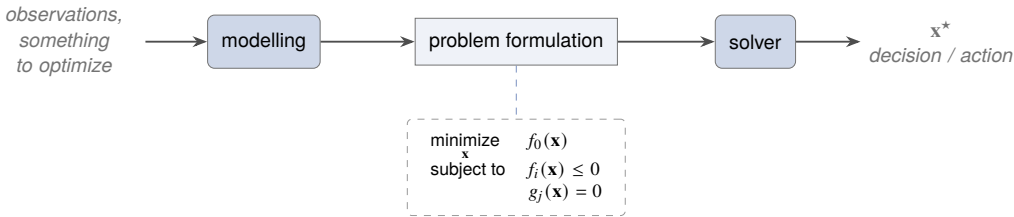
A Lightweight Canonicalization Engine in C for Disciplined Convex Programming

William Zijie Zhang

Master's Thesis Defence
October 2nd

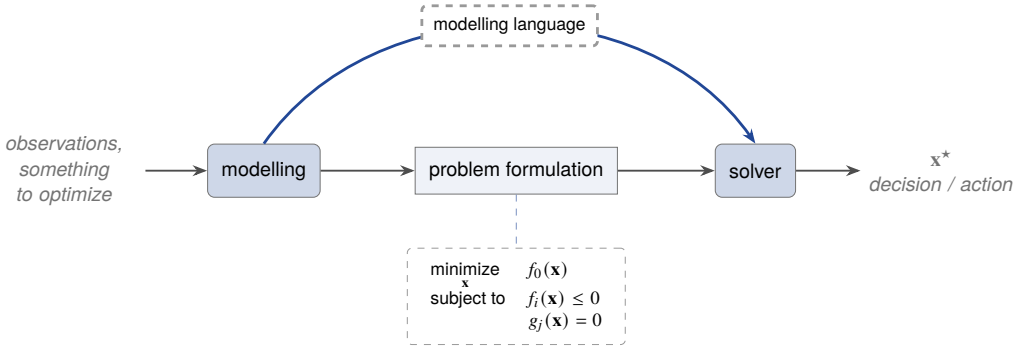
Research director: Antoine Lesage-Landry
Department of Electrical Engineering, Polytechnique Montréal

Optimization in practice



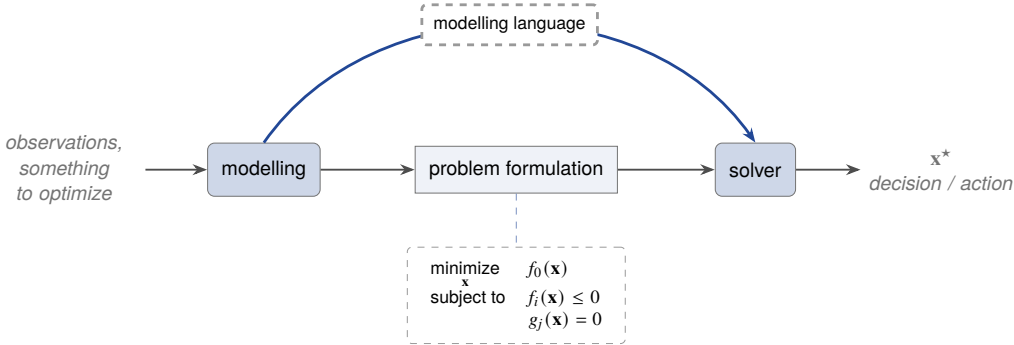
- operate a grid, control a car or a plane, manage a portfolio, design a circuit
- the formulation says what you want, not how to compute it

Optimization in practice



- operate a grid, control a car or a plane, manage a portfolio, design a circuit
- the formulation says what you want, not how to compute it
- the modelling language is the bridge between the formulation and the solver

Optimization in practice



- operate a grid, control a car or a plane, manage a portfolio, design a circuit
- the formulation says what you want, not how to compute it
- the modelling language is the bridge between the formulation and the solver
- not in this talk: how to model a problem in a domain, and how a solver works

Outline

Introduction

Example

Sparse Differentiation Engine

Results

Conclusion

Convex optimization

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f_0(\mathbf{x}) \\ \text{subject to} \quad & f_i(\mathbf{x}) \leq 0 \\ & g_j(\mathbf{x}) = 0 \end{aligned}$$

- variable $\mathbf{x} \in \mathbb{R}^n$
- the equality constraints g_j are affine, and f_0 and the f_i are convex
- why care about convexity?

Boyd and Vandenberghe (2004).

Convex optimization

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f_0(\mathbf{x}) \\ \text{subject to} \quad & f_i(\mathbf{x}) \leq 0 \\ & g_j(\mathbf{x}) = 0 \end{aligned}$$

- variable $\mathbf{x} \in \mathbb{R}^n$
- the equality constraints g_j are affine, and f_0 and the f_i are convex
- why care about convexity?
  - classic answer: a local optimum is a global optimum
  - modern answer: very actionable, because of reliable software

Boyd and Vandenberghe (2004).

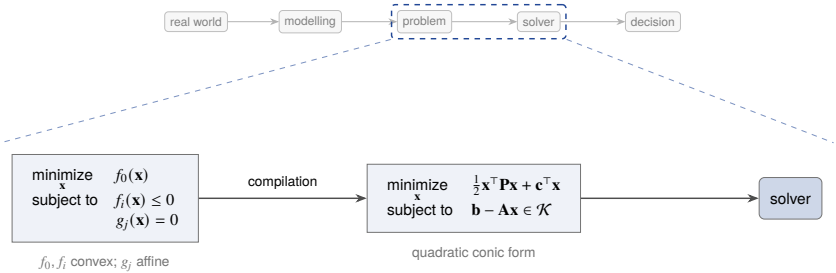
Conic form

$$\begin{array}{ll} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} & \frac{1}{2}\mathbf{x}^\top\mathbf{P}\mathbf{x} + \mathbf{c}^\top\mathbf{x} \\ \text{subject to} & \mathbf{b} - \mathbf{A}\mathbf{x} \in \mathcal{K} \end{array}$$

- problem data: $\mathbf{P} \in \mathbb{S}^n_+$, $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{c} \in \mathbb{R}^n$
- $\mathcal{K} \subseteq \mathbb{R}^m$ is a product of standard cones
- all general-purpose solvers target this standard form

Ben-Tal and Nemirovski (2001).

Convex optimization in practice



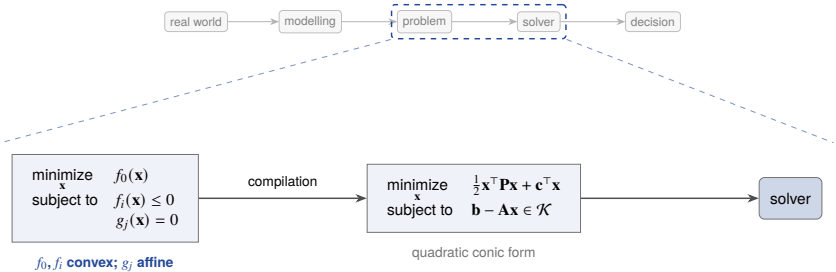
algorithms

interior point, simplex (LP), operator splitting, active set (QP)

solvers

commercial: Gurobi, MOSEK, COPT
open source: Clarabel, SCS, HiGHS

Convex optimization in practice



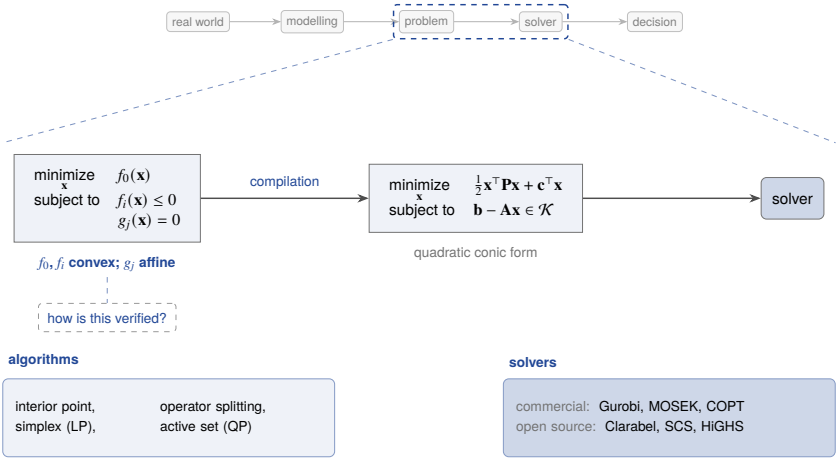
algorithms

interior point, operator splitting,
simplex (LP), active set (QP)

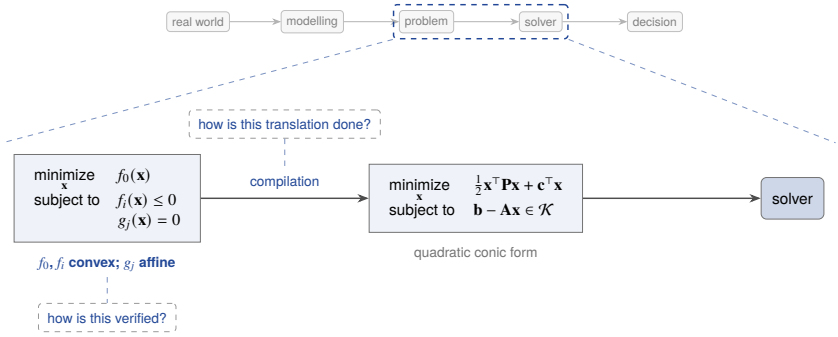
solvers

commercial: Gurobi, MOSEK, COPT
open source: Clarabel, SCS, HiGHS

Convex optimization in practice



Convex optimization in practice



Disciplined convex programming

- a grammar for specifying convex optimization problems
- convex and concave functions are built by combining a set of **atoms**
- each atom has known curvature, monotonicity, and sign
- simple rules specify how atoms can be composed

Grant, Boyd and Ye (2006).

Disciplined convex programming

- a grammar for specifying convex optimization problems
- convex and concave functions are built by combining a set of **atoms**
- each atom has known curvature, monotonicity, and sign
- simple rules specify how atoms can be composed

- a problem that follows the DCP rules is convex by construction
- and can be solved globally and efficiently — in theory and in practice

Grant, Boyd and Ye (2006).

DCP-based modelling languages

- `CVX` (MATLAB), `Convex.jl` (Julia), `CVXR` (R), `YALMIP` (MATLAB), `CVXPY` (Python)

Grant and Boyd (2008); Udell et al. (2014); Fu et al. (2020); Löfberg (2004); Diamond and Boyd (2016).

DCP-based modelling languages

- `CVX` (MATLAB), `Convex.jl` (Julia), `CVXR` (R), `YALMIP` (MATLAB), `CVXPY` (Python)
- the two tasks of a DCP-based modelling language:
  - **analysis**: verify that a high-level problem description follows the DCP rules
  - **synthesis**: build an equivalent problem in conic form, called canonicalization

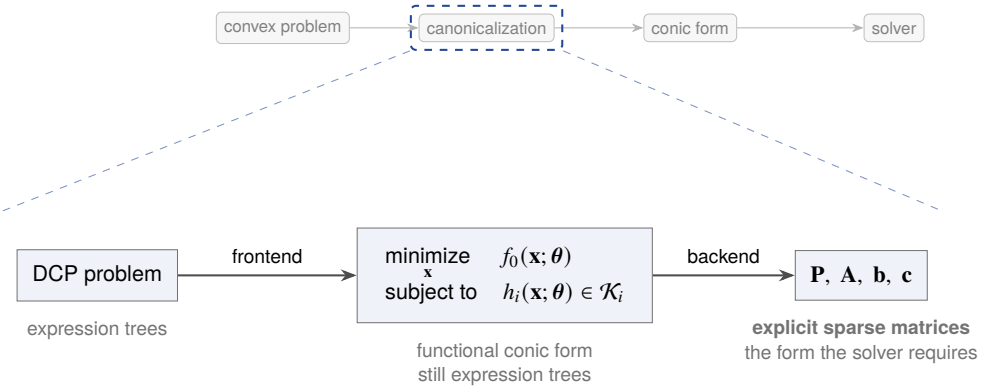
Grant and Boyd (2008); Udell et al. (2014); Fu et al. (2020); Löfberg (2004); Diamond and Boyd (2016).

DCP-based modelling languages

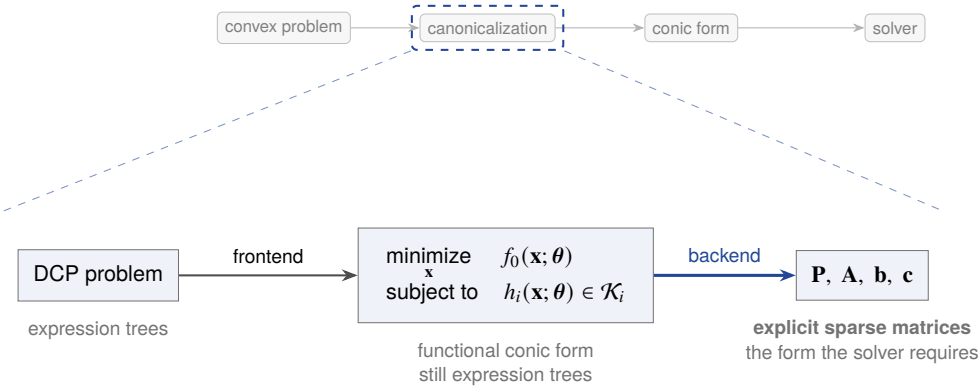
- `CVX` (MATLAB), `Convex.jl` (Julia), `CVXR` (R), `YALMIP` (MATLAB), `CVXPY` (Python)
- the two tasks of a DCP-based modelling language:
  - **analysis**: verify that a high-level problem description follows the DCP rules
  - **synthesis**: build an equivalent problem in conic form, called canonicalization

Grant and Boyd (2008); Udell et al. (2014); Fu et al. (2020); Löfberg (2004); Diamond and Boyd (2016).

Canonicalization

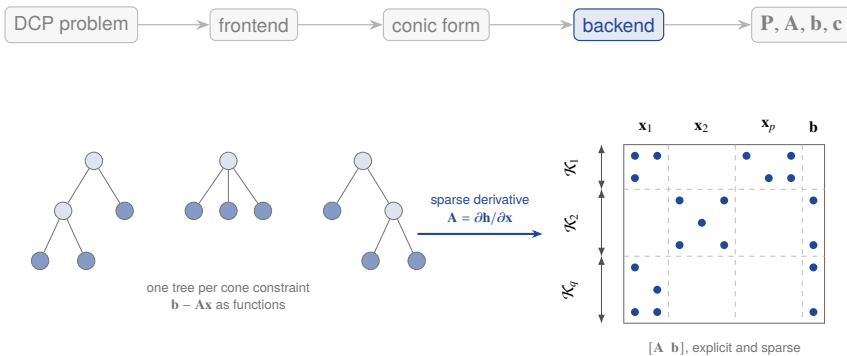


Canonicalization



- Our focus is on improving the canonicalization backend of CVXPY

Problem statement



- left: expression tree representation with many different constraints (rows) and variables (columns).
- right: one single sparse constraint matrix with all variables stacked together.

Outline

Introduction

Example

Sparse Differentiation Engine

Results

Conclusion

Fitting with regularization



$$\begin{array}{ll} \text{minimize} & \|\mathbf{F}\mathbf{x} - \mathbf{g}\|_2 + \mu\|\mathbf{x}\|_2 \\ \text{subject to} & \mathbf{x} \geq \mathbf{0} \end{array}$$

- variable $\mathbf{x} \in \mathbb{R}^n$, data $\mathbf{F} \in \mathbb{R}^{m \times n}$, parameters $\mathbf{g} \in \mathbb{R}^m$ and $\mu \geq 0$
- similar to least-squares
- physical constraint such as non-negativity of the solution

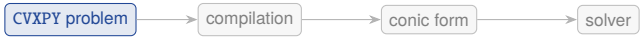
Atoms



Some atoms in the CVXPY library:

Atom	Definition	Curvature	Monotonicity	Sign
`sum`	$\sum_i x_i$	affine	non-decreasing	any
`abs`	$\lvert x \rvert$	convex	unknown	nonnegative
`square`	x^2	convex	unknown	nonnegative
`sqrt`	$\sqrt{x}$	concave	non-decreasing	nonnegative
`norm`	$\lVert \mathbf{x} \rVert_2$	convex	unknown	nonnegative
`mul`	$\boldsymbol{\mu}\mathbf{x},\ \boldsymbol{\mu} \geq 0$	affine	non-decreasing	any

A CVXPY example




```
import cvxpy as cp
import numpy as np

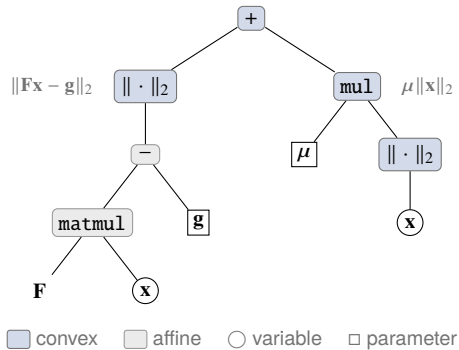
F  = np.random.randn(m, n)
x  = cp.Variable(n)
g  = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)

objective = cp.Minimize(
    cp.norm(F @ x - g) + mu * cp.norm(x))
prob = cp.Problem(objective, [x >= 0])
```

Expression tree



CVXPY stores the objective and each constraint as an expression tree:



Canonicalization: the frontend



$$\begin{array}{ll} \underset{t_1,\, t_2,\, \mathbf{x}}{\text{minimize}} & t_1 + \mu t_2 \\ \text{subject to} & \mathbf{x} \geq \mathbf{0} \\ & (t_1,\ \mathbf{F}\mathbf{x} - \mathbf{g}) \in \mathcal{K}_{\text{soc},\, m+1} \\ & (t_2,\ \mathbf{x}) \in \mathcal{K}_{\text{soc},\, n+1} \end{array}$$

$$\mathcal{K}_{\text{soc},\, k} = \{(u, \mathbf{v}) \in \mathbb{R} \times \mathbb{R}^{k-1} : \|\mathbf{v}\|_2 \leq u\}, \qquad \mathcal{K} = \mathbb{R}^n_+ \times \mathcal{K}_{\text{soc},\, m+1} \times \mathcal{K}_{\text{soc},\, n+1}$$

- the nonlinear norms are replaced by new variables t_1, t_2
- every constraint is an affine function belonging to a cone

Canonicalization: the backend



The conic form needs one stacked variable, $\tilde{\mathbf{x}} = (t_1, t_2, \mathbf{x})$:

$$\begin{aligned} &\text{minimize} && t_1 + \mu t_2 && = \mathbf{c}^\top \tilde{\mathbf{x}} \\ &\text{subject to} && (\mathbf{x},\ t_1,\ \mathbf{F}\mathbf{x} - \mathbf{g},\ t_2,\ \mathbf{x}) && = \mathbf{b} - \mathbf{A}\tilde{\mathbf{x}} \in \mathcal{K} \end{aligned}$$

$$\mathbf{c}^\top = \begin{matrix} t_1 & t_2 & \mathbf{x} \end{matrix} \begin{bmatrix} 1 & \mu & 0 \end{bmatrix} \qquad [\mathbf{A}\ \ \mathbf{b}] = \begin{matrix} t_1 \\ \\ \\ \\ \\ \\ \end{matrix} \begin{matrix} t_2 \\ \\ \\ \\ \\ \\ \end{matrix} \begin{matrix} \mathbf{x} \\ \\ \\ \\ \\ \\ \end{matrix} \begin{matrix} \mathbf{b} \\ \\ \\ \\ \\ \\ \end{matrix} \begin{bmatrix} 0 & 0 & -\mathbf{I} & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & -\mathbf{F} & -\mathbf{g} \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -\mathbf{I} & 0 \end{bmatrix}$$

Parameters in CVXPY



Solving the same problem for different values of the parameters

```
g  = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)
mus = [1, 10, 25, 100]

for _ in range(trials):
    g.value = np.random.randn(m)

    for mu_value in mus:
        mu.value = mu_value
        prob.solve()
```

- CVXPY can reuse the problem instance for different parameter values

Parameters in the CVXPY backend



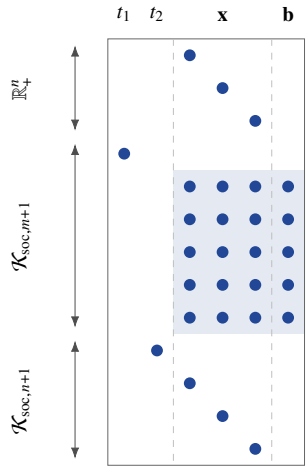
$$\begin{aligned} &\text{minimize} && t_1 + \mu\, t_2 && = \mathbf{c}^\top \tilde{\mathbf{x}} \\ &\text{subject to} && (\mathbf{x},\ t_1,\ \mathbf{F}\mathbf{x} - \mathbf{g},\ t_2,\ \mathbf{x}) && = \mathbf{b} - \mathbf{A}\tilde{\mathbf{x}} \in \mathcal{K} \end{aligned}$$

$$\mathbf{c}^\top = \begin{matrix} t_1 & t_2 & \mathbf{x} \end{matrix} \begin{bmatrix} 1 & \mu & 0 \end{bmatrix} \qquad [\mathbf{A}\ \ \mathbf{b}] = \begin{matrix} t_1 & t_2 & \mathbf{x} & \mathbf{b} \end{matrix} \begin{bmatrix} 0 & 0 & -\mathbf{I} & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & -\mathbf{F} & -\mathbf{g} \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -\mathbf{I} & 0 \end{bmatrix}$$

- the structure of the problem stays fixed: a zero stays a zero
- most of the problem data is constant

Structure in the problem data

convex problem → compilation → conic form → solver



$[\mathbf{A} \;\; \mathbf{b}]$ at $n = 3$, $m = 5$

- most optimization problems have similar structure
- exploiting structure while computing a sparse derivative is not straightforward

Outline

Introduction

Example

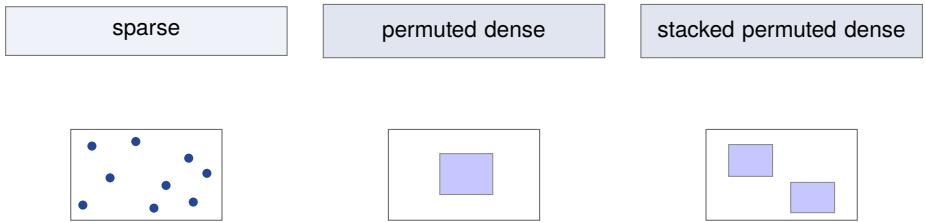
Sparse Differentiation Engine

Results

Conclusion

Overview

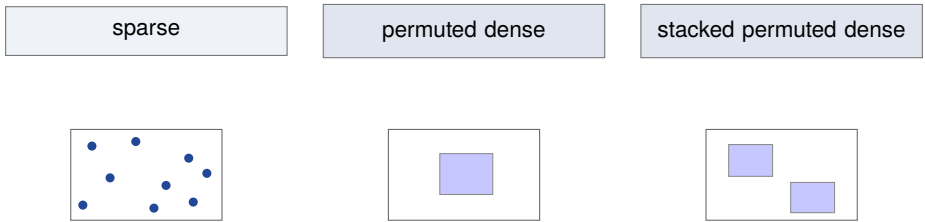
`SparseDiffEngine` is a small C library that computes sparse Jacobians and Hessians from the expression trees of `CVXPY` atoms.



The goal is to use different representations to exploit the different structures in the problem data.

Overview

`SparseDiffEngine` is a small C library that computes sparse Jacobians and Hessians from the expression trees of `CVXPY` atoms.



The goal is to use different representations to exploit the different structures in the problem data.

- problem level: the canonicalized data is mostly zeros
- block level: each data matrix enters as a dense block inside it

Compressed sparse row (CSR) matrix

$$\mathbf{S} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 2 & 0 & 3 & 4 \\ 0 & 5 & 6 & 0 \end{bmatrix}$$

```
int    idxptr[] = {0,  1,          4,      6};
int      cols[] = {1,  0,  2,  3,  1,  2};
double   vals[] = {1,  2,  3,  4,  5,  6};
```

- CVXPY stores all problem data as CSR matrices
- Compressed sparse column (CSC) matrices are preferred for column-oriented operations
- CSR/CSC matrices are the format that most solvers expect

Permuted-Dense matrix

$$\mathbf{A} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & b_{00} & \cdot & \cdot & b_{01} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{10} & \cdot & \cdot & b_{11} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{20} & \cdot & \cdot & b_{21} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{30} & \cdot & \cdot & b_{31} & \cdot & \cdot \end{pmatrix} \qquad \mathbf{X} = \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \\ b_{20} & b_{21} \\ b_{30} & b_{31} \end{pmatrix}$$

$$\mathbf{r} = (0, 2, 3, 5), \qquad \mathbf{c} = (4, 7), \qquad A_{r_i, c_j} = X_{ij}$$

A permuted dense matrix is a dense submatrix after applying some row and column selections.

Permuted-Dense matrix

$$\mathbf{A} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & b_{00} & \cdot & \cdot & b_{01} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{10} & \cdot & \cdot & b_{11} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{20} & \cdot & \cdot & b_{21} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{30} & \cdot & \cdot & b_{31} & \cdot & \cdot \end{pmatrix} \qquad \mathbf{X} = \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \\ b_{20} & b_{21} \\ b_{30} & b_{31} \end{pmatrix}$$

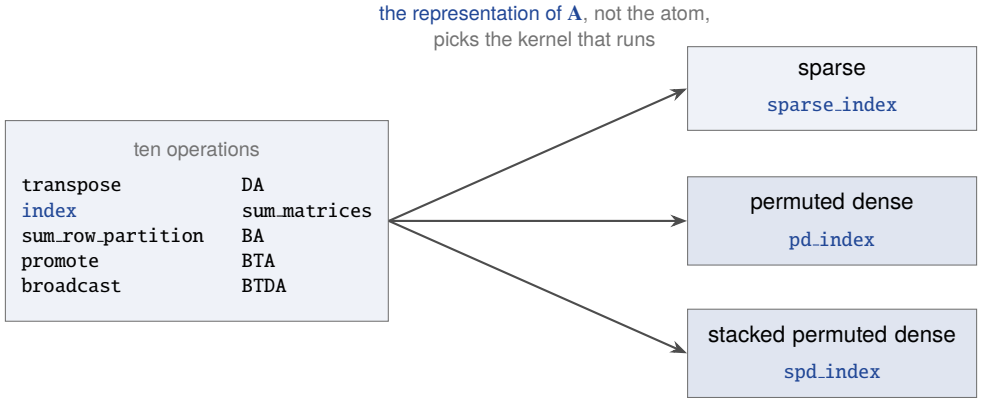
$$\mathbf{r} = (0, 2, 3, 5), \qquad \mathbf{c} = (4, 7), \qquad A_{r_i, c_j} = X_{ij}$$

A permuted dense matrix is a dense submatrix after applying some row and column selections.

- unlocks dense linear algebra storage and operations.
- BLAS: basic linear algebra subprograms are highly optimized dense operations.
- the stacked version can exploit different blocks at disjoint rows and columns.

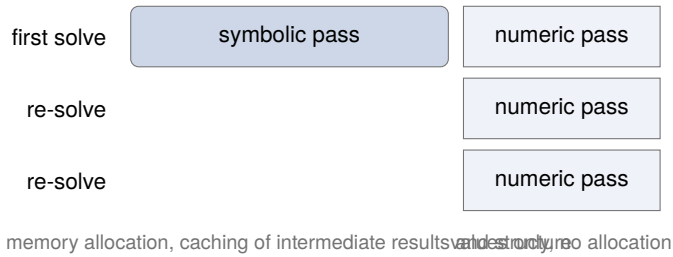
Lawson et al. (1979); Dongarra et al. (1990).

Algorithms



- $\mathbf{A}$ is a Jacobian; $\mathbf{B}$ and $\mathbf{D} = \operatorname{diag}(\mathbf{d})$ are problem data
- ten affine operations cover most of the atom library
- each operation has a corresponding kernel for every matrix representation.

Canonicalization in two phases



- re-solving a parametrized problem costs one numeric pass

Outline

Introduction

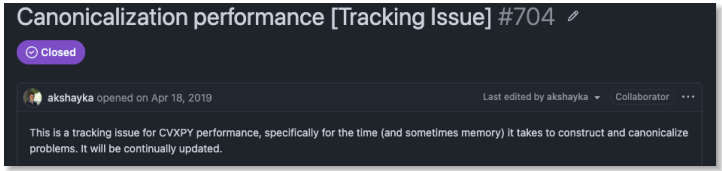
Example

Sparse Differentiation Engine

Results

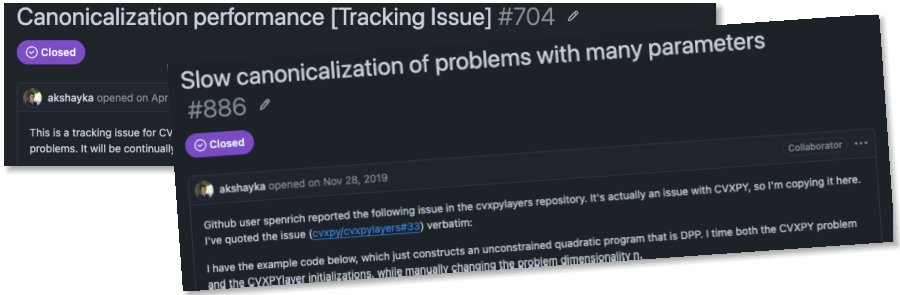
Conclusion

Why do we care about canonicalization?



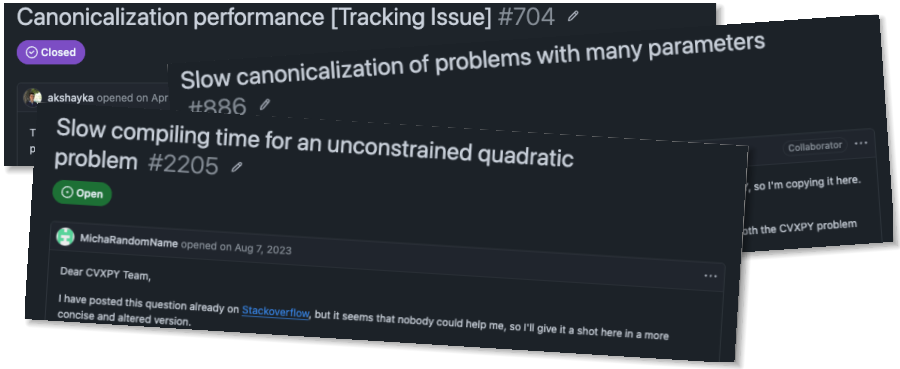
`github.com/cvxpy/cvxpy` — issues #704, #886, #2205, #3072.

Why do we care about canonicalization?



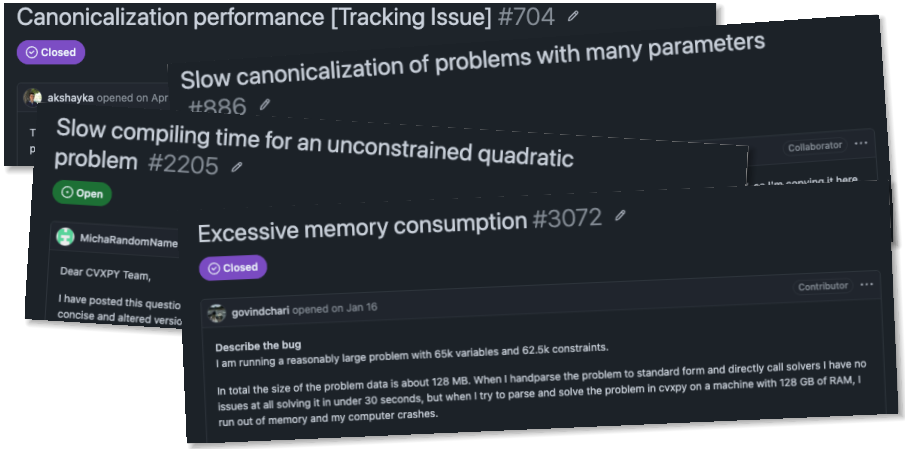
`github.com/cvxpy/cvxpy` — issues #704, #886, #2205, #3072.

Why do we care about canonicalization?



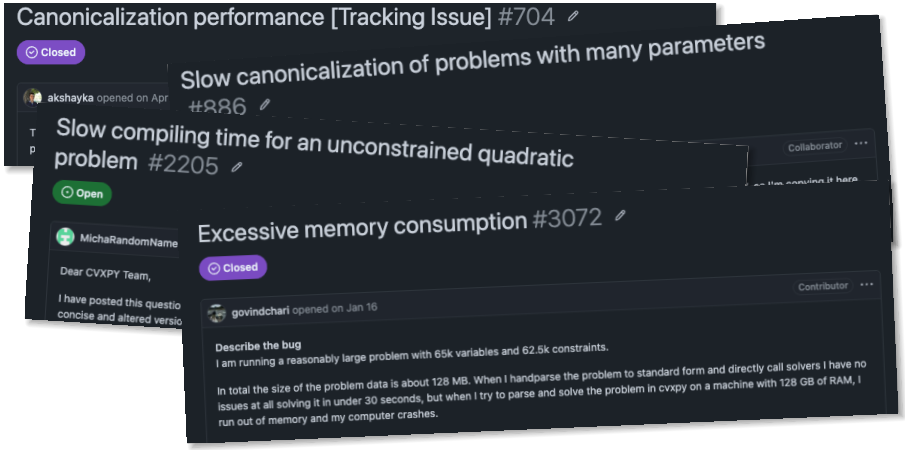
`github.com/cvxpy/cvxpy` — issues #704, #886, #2205, #3072.

Why do we care about canonicalization?



`github.com/cvxpy/cvxpy` — issues #704, #886, #2205, #3072.

Why do we care about canonicalization?



How well does our approach improve the performance of canonicalization?

`github.com/cvxpy/cvxpy` — issues #704, #886, #2205, #3072.

Benchmark setup

The 25 problems of CVXPY's public benchmark suite: portfolio selection, regression and classification, image inpainting, and many more applications of convex optimization.

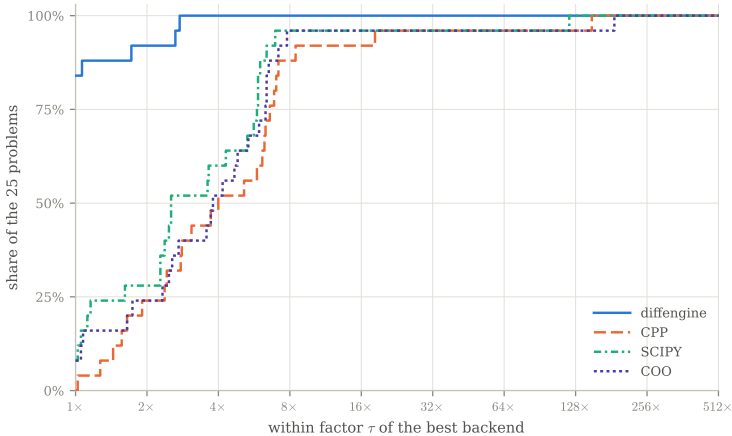
```
data, _, _ = problem.get_problem_data(
    solver=cp.CLARABEL,
    canon_backend="DIFFENGINE",   # CPP | SCIPY | COO
    ignore_dpp=True)
```

CPP (cvxcore)	SCIPY	COO	diffengine
C++, the default	Python	Python	C

Single-threaded, 2018 MacBook Pro / Core i7-8850H • 20-minute limit • CVXPY benchmarks (2025), github.com/cvxpy/benchmarks.

Performance profile

Share of the 25 problems on which a backend runs within a factor τ of the fastest backend on that problem:

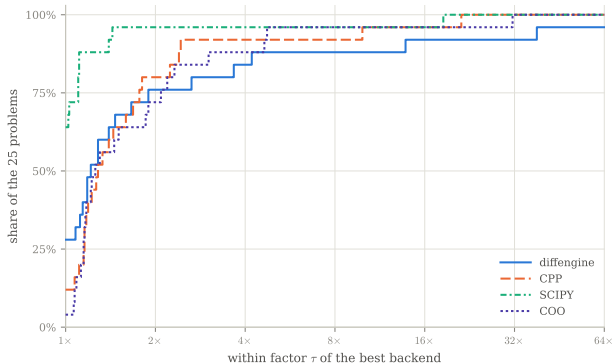


`diffengine` is within $3\times$ of the best backend on *every* problem.

Fastest of the four on 21 of the 25, and more than twice as fast as the best `CVXPY` backend on 17.

Peak memory

The same profile with $t_{p,s}$ replaced by the growth of the process's peak resident memory during canonicalization:



SCIPY has the lowest peak memory on 16 of the 25 problems.

diffengine matches CPP and COO in geometric mean, and uses about 1.4× the memory of SCIPY.

Outline

Introduction

Example

Sparse Differentiation Engine

Results

Conclusion

Takeaways

What CVXPY does between `cp.Problem` and the solver — and what it costs.

From a lot of time spent in the backend:

- use a `cp.Parameter` when the same problem is solved again
- vectorize — one operation rather than a loop, where it does not cost readability
- read the release notes: the library keeps growing features that do this for you

Takeaways

What CVXPY does between `cp.Problem` and the solver — and what it costs.

From a lot of time spent in the backend:

- use a `cp.Parameter` when the same problem is solved again
- vectorize — one operation rather than a loop, where it does not cost readability
- read the release notes: the library keeps growing features that do this for you

```
rows, cols = pattern.nonzero()
X = cp.Variable((n, n), sparsity=(rows, cols))
```

one scalar variable per stored entry, not n^2.

Conclusion

- not covered today: what CVXPY does instead — the parameter-to-data tensor, and what the split costs against it
- ahead: solvers that accept, or mix, matrix representations, so the structure survives canonicalization into the solve itself

Conclusion

- not covered today: what CVXPY does instead — the parameter-to-data tensor, and what the split costs against it
- ahead: solvers that accept, or mix, matrix representations, so the structure survives canonicalization into the solve itself

The structure is in the problem already. The engine is what keeps it.

Thank you

References

Agrawal, A., Amos, B., Barratt, S., Boyd, S., Diamond, S. and Kolter, Z. (2019). Differentiable convex optimization layers. *Advances in Neural Information Processing Systems*, 32.

Ben-Tal, A. and Nemirovski, A. (2001). *Lectures on Modern Convex Optimization: Analysis, Algorithms, and Engineering Applications*. SIAM.

Boyd, S. and Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

The CVXPY developers (2025). CVXPY benchmarks. `https://github.com/cvxpy/benchmarks`

Diamond, S. and Boyd, S. (2016). CVXPY: a Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5.

Dolan, E. D. and Moré, J. J. (2002). Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91(2):201–213.

Dongarra, J. J., Du Croz, J., Hammarling, S. and Duff, I. S. (1990). A set of Level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software*, 16(1):1–17.

Fu, A., Narasimhan, B. and Boyd, S. (2020). CVXR: an R package for disciplined convex optimization. *Journal of Statistical Software*, 94(14).

References (continued)

Grant, M. and Boyd, S. (2008). Graph implementations for nonsmooth convex programs. In *Recent Advances in Learning and Control*, Lecture Notes in Control and Information Sciences 371. Springer.

Grant, M., Boyd, S. and Ye, Y. (2006). Disciplined convex programming. In *Global Optimization: From Theory to Implementation*, pages 155–210. Springer.

Gustavson, F. G. (1978). Two fast algorithms for sparse matrices: multiplication and permuted transposition. *ACM Transactions on Mathematical Software*, 4(3):250–269.

Lawson, C. L., Hanson, R. J., Kincaid, D. R. and Krogh, F. T. (1979). Basic linear algebra subprograms for Fortran usage. *ACM Transactions on Mathematical Software*, 5(3):308–323.

Löfberg, J. (2004). YALMIP: a toolbox for modeling and optimization in MATLAB. *IEEE International Conference on Robotics and Automation*, pages 284–289.

Udell, M., Mohan, K., Zeng, D., Hong, J., Diamond, S. and Boyd, S. (2014). Convex optimization in Julia. *First Workshop for High Performance Technical Computing in Dynamic Languages*, pages 18–28.

Backup: the DCP rules

Every atom declares its curvature, its monotonicity in each argument, and the sign of its output:

Atom	Definition	Curvature	Monotonicity	Sign
`sum`	$\sum_i x_i$	affine	non-decreasing	any
`abs`	$\lvert x \rvert$	convex	unknown	nonnegative
`square`	x^2	convex	unknown	nonnegative
`sqrt`	$\sqrt{x}$	concave	non-decreasing	nonnegative
`norm`	$\lVert \mathbf{x} \rVert_2$	convex	unknown	nonnegative

The composition $\phi(u_1(\mathbf{x}), \ldots, u_k(\mathbf{x}))$ is convex when ϕ is convex and, for each argument i, one of the following holds:

- ϕ is non-decreasing in i and u_i is convex;
- ϕ is non-increasing in i and u_i is concave; or
- u_i is affine.

Swap convex and concave for the concave rule. A problem is DCP-compliant when the objective and every constraint come out compatible: convex objective, convex $\leq$ concave, affine equalities.

The rules are sufficient for convexity, not necessary: a genuinely convex problem written in an unrecognized form is rejected.

Backup: sufficient, not necessary

The same function, written two ways. Only one of them compiles.

$$\sqrt{x^2 + 1} \;=\; \left\|(x,\ 1)\right\|_2$$

```
cp.sqrt(cp.square(x) + 1)
```

rejected

- `sqrt` is concave and non-decreasing
- its argument $x^2 + 1$ is convex, not affine
- no rule covers concave $\circ$ convex, so the curvature is unknown

```
cp.norm(cp.hstack([x, 1]))
```

accepted

- `norm` is convex
- its argument $(x, 1)$ is affine
- convex $\circ$ affine is convex, and the epigraph is $(t,\ x,\ 1) \in \mathcal{K}_{\text{soc},3}$

The rules are a test on the expression, not on the function. Rewriting is the user's job; the atom library is what makes it possible.

Backup: why CVXPY

- open source, embedded in Python, the most widely used programming language
- in development for over ten years, over 6 million downloads a month from PyPI
- 4.5 k citations of the 2016 CVXPY paper

TIOBE index, pypistats.org, github.com/cvxpy/cvxpy and Google Scholar, September 2026.

Backup: disciplined parametrized programming

DPP is a second grammar, on top of DCP, restricting how parameters may enter an expression: every parameter is treated as affine, and products of two parametrized expressions are forbidden.

The payoff is that the problem data becomes affine in the parameters:

$$\mathbf{c} = \mathbf{Q}\,\tilde{\boldsymbol{\theta}}, \qquad \operatorname{vec}([\mathbf{A}\ \mathbf{b}]) = \mathbf{R}\,\tilde{\boldsymbol{\theta}}, \qquad \tilde{\boldsymbol{\theta}} = \begin{bmatrix} \boldsymbol{\theta} \\ 1 \end{bmatrix}$$

- `CVXPY` builds $\mathbf{Q}$ and $\mathbf{R}$ by adding a third dimension to every coefficient matrix of the pass, indexed by the parameter entries
- after a parameter change, a re-solve costs one sparse matrix-vector product per map: no tree is traversed again
- a problem outside the grammar, or one passing `ignore_dpp`, takes the `EvalParams` path instead: each parameter leaf is replaced by its current value and the whole canonicalization repeats

This is the baseline the parametric results are measured against.

Agrawal et al. (2019).

Backup: two ways to get the coefficients

Every modelling language reviewed produces the same output, the solver problem data. They split into two families by what they ask of the user.

Differentiate an expression graph

- `CasADi`: built for nonlinear optimization; its conic interface `qpsol` differentiates the graph through matrix-vector products, so the sparsity pattern must be detected at runtime and compressed by colouring
- `CVXPY`: composes the linear operators of the conic-form trees in one bottom-up pass, which is equivalent to differentiating them; expressions stay vectorized, so no detection step is needed
- `SparseDiffEngine`: differentiates them outright, one Jacobian per node with respect to *all* the variables, so the chain rule is a single matrix product — not a dictionary of blocks keyed by variable

Read the coefficients the user declares

- `AMPL`, `JuMP`: the user writes each objective and constraint as a sum of terms, and every linear and quadratic coefficient is stored as it is typed
- neither has an atom library or a canonicalization step, so a norm is left to the solver (`AMPL`) or written in conic form by hand (`JuMP`); both scalarize

Backup: the research gap

System	Expression storage	Coefficients obtained by	Reused across the family	Repeated at every solve
`AMPL`	scalar	instance generation	nothing, values folded into the instance	`.nl` write and read
`JuMP`	scalar	term lists stored by macros	term lists, modified in place	`copy_to` into the solver
`CasADi`	scalar or matrix	differentiation of an expression graph	pattern, colouring, derivative function	one sweep per colour
`CVXPY` backends	vectorized	composition of linear operators	parameter-to-data tensor, DPP problems only	one tensor-vector product, or the whole pass
`SparseDiffEngine`	vectorized	structured direct differentiation	problem structure, by the symbolic pass	numeric pass

None of the four combines the two-phase split with direct computation of the problem data.

Two phases like `CasADi`, but the whole coefficient matrix at once instead of matrix-vector products, and dense storage where the structure is dense.

Backup: when the variable is a matrix

Multiplying a matrix variable $\mathbf{V} \in \mathbb{R}^{2\times 2}$ by dense $\mathbf{B}$ acts on each column separately: one dense block per column of $\mathbf{V}$, at disjoint rows and different columns.

$$\mathbf{M} = \begin{pmatrix} \cdot & \cdot & \cdot & \cdot & b_{00} & b_{01} & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & b_{10} & b_{11} & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & b_{00} & b_{01} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & b_{10} & b_{11} & \cdot & \cdot \end{pmatrix}$$

$$\mathbf{X}_1 = \mathbf{X}_2 = \mathbf{B}, \qquad \mathbf{r}_1 = (0,1),\ \mathbf{c}_1 = (4,5), \qquad \mathbf{r}_2 = (2,3),\ \mathbf{c}_2 = (6,7)$$

- no single triple describes this: merging the blocks would store the zeros between them, and the sparse form would lose the dense structure
- a stacked permuted dense matrix is the vertical union of blocks at pairwise disjoint rows — every operation treats them one at a time, and the global matrix is never formed
- the stack owns one value buffer and each block points into it, so $\mathbf{B}$ is stored once however many columns $\mathbf{V}$ has

Backup: the ten kernels

Ten operations cover the whole atom library, each implemented once per representation, as a symbolic-pass and a numeric-pass function.

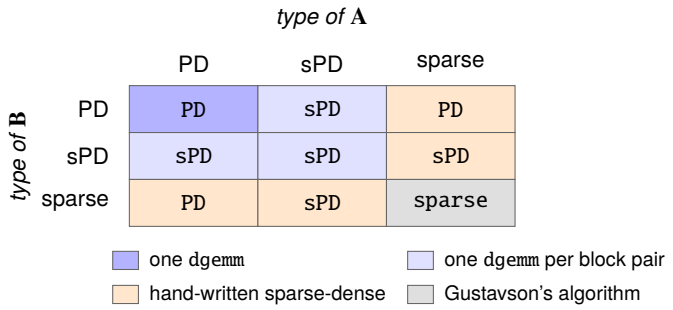
Expression	Kernel
`index`, `transpose`	`index`
`sum`	`sum_row_partition`
`broadcast`, `promote`	`broadcast`, `promote`
`vector_mult`	`DA`
`add`	`sum_matrices`
`left_matmul`	`BA`, `transpose`
`quad_form`	`BTA`, `BTDA`

- no per-atom derivative code: every atom reaches its Jacobian through one of these
- each kernel is written once per representation — `index` copies column indices on CSR, intersects the row vector on a dense block, and drops the empty blocks of a stacked one
- the dense form is absorbing: in a product, a sparse operand never flattens a block

Backup: which kernel runs on which pair

PD is a dense block at permuted positions $(\mathbf{X}, \mathbf{r}, \mathbf{c})$; sPD is the stack of such blocks of the previous slide. Written out, the product of two permuted dense matrices is

$$\mathbf{B}^\top \mathbf{A} = \mathbf{S}_{c_B}^\top \mathbf{X}_B^\top \underbrace{\left(\mathbf{S}_{r_B} \mathbf{S}_{r_A}^\top\right)}_{\text{index-set intersection}} \mathbf{X}_A \mathbf{S}_{c_A}.$$



Eight of the nine kernels come out dense; cost depends on block size, not on m and n.

Backup: addition is where the dense path stops

- the Jacobian of $\mathbf{F}\mathbf{x} + \mathbf{y}$ holds the block of $\mathbf{F}$ at the columns of $\mathbf{x}$ *and* an identity at the columns of $\mathbf{y}$
- no single triple $(\mathbf{X}, \mathbf{r}, \mathbf{c})$ covers both, so the sum is performed on CSR views of the two operands instead
- the views are built during the symbolic pass and their value arrays point at the dense blocks, so no values are copied
- the result is a sparse matrix, and so is everything computed from it

The dense representations are sticky in the other direction: a product with a permuted dense or stacked operand gives back a matrix of the same kind.

Backup: a problem we do not win

`SDPSegfault1132`, one of the two problems of the suite where `diffengine` is behind

```
n = 100
V = ...                         # dense, (n, n-1)
D = ...                         # dense, (n, n)
W = np.ones((n, n))             # dense, (n, n)

G = cp.Variable((n-1, n-1), PSD=True)
A = cp.kron(e,   cp.reshape(cp.diag(V @ G @ V.T), (1, n)))
B = cp.kron(e.T, cp.reshape(cp.diag(V @ G @ V.T), (n, 1)))
C = cp.norm(cp.multiply(W, A + B - 2 * V @ G @ V.T - D), p='fro')

prob = cp.Problem(cp.Maximize(cp.trace(G) - C))
```

- every constant is dense, and `V @ G @ V.T` sandwiches the variable between two of them
- `diffengine` takes 13.21 s against SCIPY's 9.51 s: processing the dense constant data dominates

Backup: what the two-phase split costs

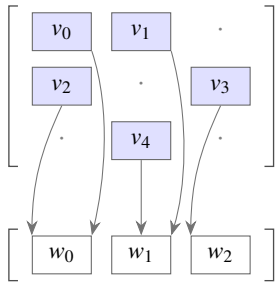
- the numeric pass re-evaluates the expression trees at every parameter change, so the `problem` object must stay in memory for the lifetime of the problem family
- the parameter-to-data tensor has no such constraint: once it is built, a re-solve needs only its product with the current parameter values, and the formulation can be set aside
- a parameter update also forces the numeric pass to recompute the entire constant part of the problem data, which is why the two problems with almost-constant data and a few scalar parameters re-solve faster through the tensor

This is the one place the tensor is genuinely the better object.

Backup: what the symbolic pass stores

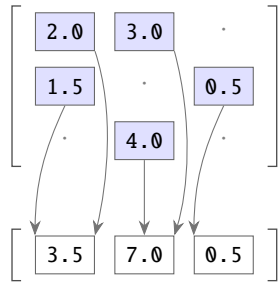
Depending on the kernel, the stored structure is a **sparsity pattern**, a **block layout**, or an **index map**.

symbolic pass, once



$\texttt{map} = (0, 1, 0, 2, 1)$

numeric pass, per evaluation



$\texttt{out}[\texttt{map}[j]]$ += $\texttt{vals}[j]$

The `sum` atom: the symbolic pass records where each nonzero lands, the numeric pass follows the fixed arrows.

`index`, `transpose` and `broadcast` build the same map, with a copy in place of an addition.

Backup: the parameter maps

Under DPP the data is affine in the parameters, so the family collapses to two cached maps:

$$\mathbf{c} = \mathbf{Q}\,\tilde{\boldsymbol{\theta}}, \qquad \operatorname{vec}([\mathbf{A}\ \mathbf{b}]) = \mathbf{R}\,\tilde{\boldsymbol{\theta}}, \qquad \tilde{\boldsymbol{\theta}} = \begin{bmatrix} \boldsymbol{\theta} \\ 1 \end{bmatrix}$$

`CVXPY`'s three backends are three representations of that same tensor:

- `cvxcore` (C++) — one sparse matrix per parameter
- `SCIPY` — the slices stacked into one strided sparse matrix
- `COO` — never formed: triples with a parameter index

Backup: parametric problems

The seven DPP parametric problems • a dash is a tensor that did not finish inside the time limit

	first canonicalization				re-solve		
Benchmark	CPP	SCIPY	C00	diffengine	tensor	diffengine	Ratio
ParamSmallMatrixStuffing	7.78	29.06	0.40	**0.21**	0.005	**0.001**	0.29
ParamConeMatrixStuffing	3.16	7.63	0.26	**0.14**	0.003	**0.001**	0.29
SimpleFullyParametrizedLP	—	—	2.17	**0.54**	0.261	**0.126**	0.48
ParametrizedQP	—	—	10.87	**2.53**	1.485	**0.985**	0.66
SimpleScalarParametrizedLP	4.68	4.15	4.48	**1.03**	0.304	**0.247**	0.81
SVMWithL1Regularization	11.00	8.21	9.62	**1.86**	**0.252**	0.496	1.97
FactorCovarianceModel	4.92	4.15	4.66	**0.86**	**0.139**	0.279	2.00

Fastest first canonicalization on every problem; re-solve within about $2\times$ the tensor.

The last two: data almost all constant, and an update still recomputes it.

Backup: the per-problem times

25 problems of CVXPY's public benchmark suite • `get_problem_data` with `Clarabel`, solver excluded • the `ignore_dpp` path • single-threaded, 20-minute limit

Benchmark	CPP (s)	SCIPY (s)	COO (s)	diffengine (s)	Ratio
UnconstrainedQP	10.32	8.52	13.54	**0.07**	0.01
LeastSquares	9.61	10.08	10.68	**1.45**	0.15
SimpleLP	37.57	29.99	30.96	**6.12**	0.20
Cajas	22.67	2.91	2.89	**1.07**	0.37
ConeMatrixStuffing	11.06	9.34	16.99	**3.95**	0.42
⋮	⋮	16 further problems	⋮	⋮	⋮
CVaR	35.85	**24.79**	26.86	24.94	1.01
OptimalAdvertising	6.43	2.32	**2.25**	2.36	1.05
SDPSegfault1132	64.84	**9.51**	43.00	13.21	1.39
ConvexPlasticity	**0.35**	**0.35**	**0.35**	0.85	2.48
Geometric mean					**0.36**

Fastest of the four on 21 of the 25 problems, and more than twice as fast as the best CVXPY backend on 17.

Backup: how to read a performance profile

A performance profile compares the backends over the whole suite at once, rather than one problem at a time.

$$f_s(\tau) = \frac{1}{n} \left| \{ p : t_{p,s} \le \tau \min_b t_{p,b} \} \right|$$

- $t_{p,s}$ is the time of backend s on problem p, and $\min_b t_{p,b}$ is the time of the fastest backend on that same problem
- $f_s(\tau)$ is the fraction of the n problems on which backend s lands within a factor τ of that best time
- at $\tau = 1$: the fraction of problems on which the backend is outright fastest
- where the curve reaches 1: the factor the backend needs in order to cover every problem in the suite

Dolan and Moré (2002).